DNS보다 먼저 확인하는 곳. Linux /etc/hosts로 직접 확인해보기

웹브라우저나 애플리케이션에서 도메인으로 서버에 접속할 때 우리는 보통 다음 흐름을 떠올린다.

```
도메인 입력
   ↓
DNS 서버에 질의
   ↓
IP 주소 확인
   ↓
해당 서버로 접속
```

하지만 Linux에서도 Windows와 마찬가지로 **모든 이름 해석이 곧바로 DNS 질의로 이어지는 것은 아니다.**

일반적인 Linux 환경에서는 애플리케이션이 호스트 이름을 IP로 바꾸려고 할 때 시스템의 이름 해석 체계를 사용한다. 이때 `/etc/nsswitch.conf`의 `hosts` 항목에 정의된 순서에 따라 `/etc/hosts` 같은 로컬 정보가 DNS보다 먼저 사용될 수 있다.

이번 실습에서는 실제 도메인 `hwangsoojin.cloud`의 DNS 레코드는 그대로 둔 채, Linux 서버의 `/etc/hosts`에 임의의 IP를 넣어 **서버 내부에서만 도메인의 목적지를 바꿔 보는 과정**을 확인한다.

> 💡 **이번 실습에서 확인할 것**
>
> - `/etc/hosts`는 Linux 서버 내부의 로컬 이름 매핑 파일이다.
> - `/etc/hosts` 조회 자체는 네트워크 통신이 아니라 **로컬 파일 조회**다.
> - `nslookup`, `dig`는 DNS 서버에 직접 질의하므로 `/etc/hosts` 변경을 반영하지 않는다.
> - `getent hosts`, `ping`, `curl`은 일반적인 시스템 이름 해석 결과를 확인하는 데 더 적합하다.

---

1. 먼저 실제 DNS에 어떤 IP가 등록되어 있는지 확인

실습 전 `hwangsoojin.cloud`의 DNS 레코드를 확인했다.

현재 DNS에는 두 개의 A 레코드가 등록되어 있다.

```
hwangsoojin.cloud → 111.111.111.111
hwangsoojin.cloud → 111.111.111.112
```



hwangsoojin.cloud에 등록되어 있던 실제 DNS A 레코드

이번 실습에서는 이 DNS 레코드 자체는 수정하지 않는다.

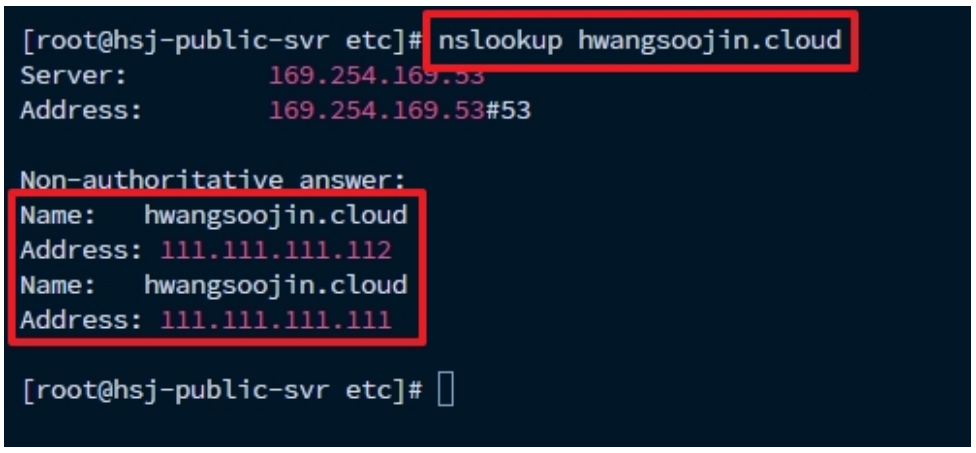
즉, 외부 사용자가 DNS 서버에 `hwangsoojin.cloud`을 질의하면 계속 위 두 IP를 받는다.

---

2. nslookup으로 DNS 질의 결과 확인

Linux 서버에서 먼저 `nslookup`을 실행했다.

```
nslookup hwangsoojin.cloud
```



nslookup으로 확인한 hwangsoojin.cloud의 DNS 응답

결과 역시 DNS에 등록된 두 IP가 반환된다.

```
111.111.111.111
111.111.111.112
```

여기서 `nslookup`은 **DNS 서버에 직접 질의해서 받은 결과를 보여주는 도구**라고 기억해 두자.

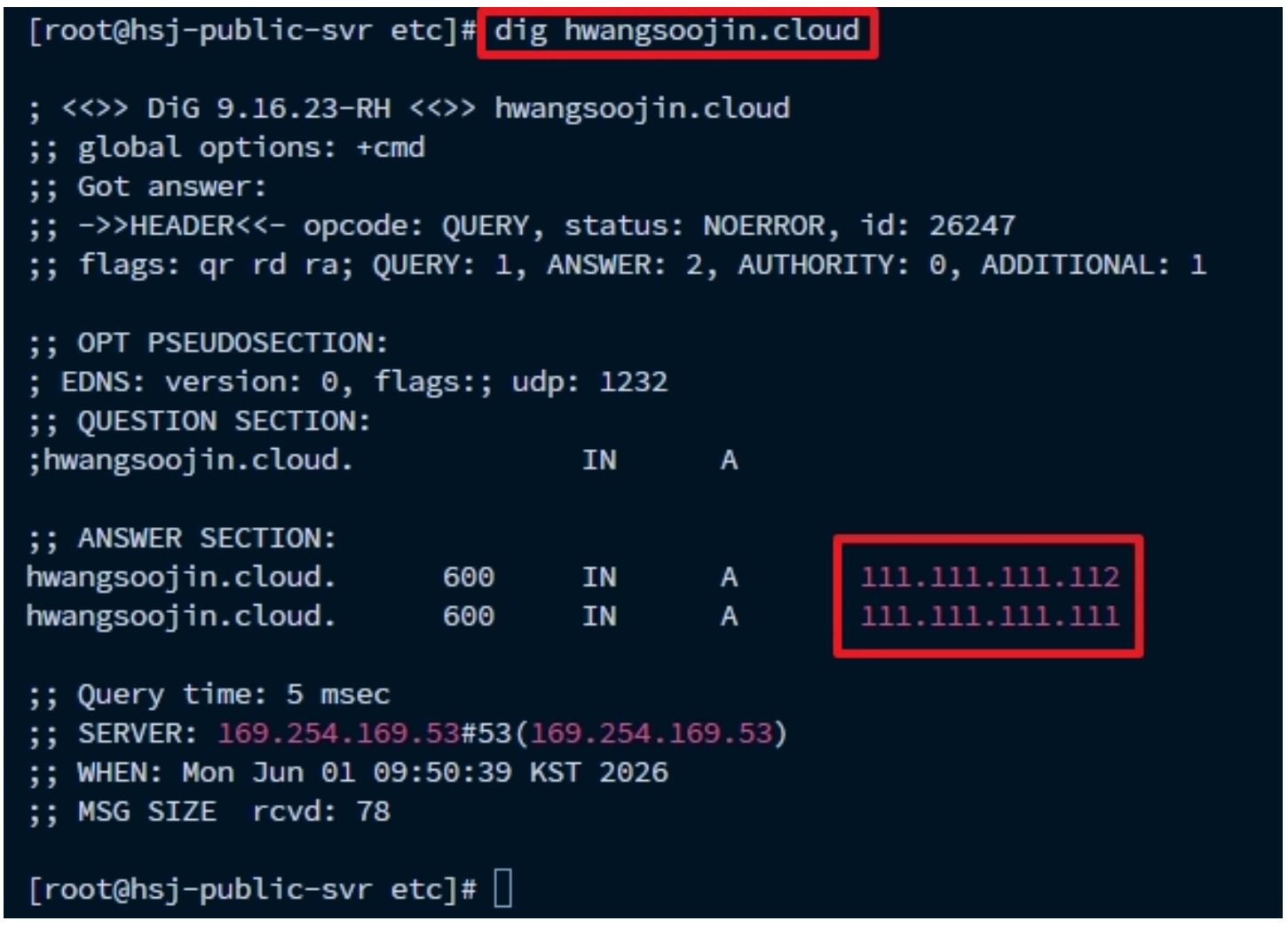
이 점이 뒤에서 중요해진다.

---

3. dig로도 DNS 응답 확인

같은 내용을 `dig`로도 확인했다.

```
dig hwangsoojin.cloud
```



dig 명령의 ANSWER SECTION에서 동일한 A 레코드를 확인한 모습

`ANSWER SECTION`에서도 동일하게 두 개의 A 레코드가 반환된다.

즉, 현재 외부 DNS 기준으로는

```
hwangsoojin.cloud
   ├─ 111.111.111.111
   └─ 111.111.111.112
```

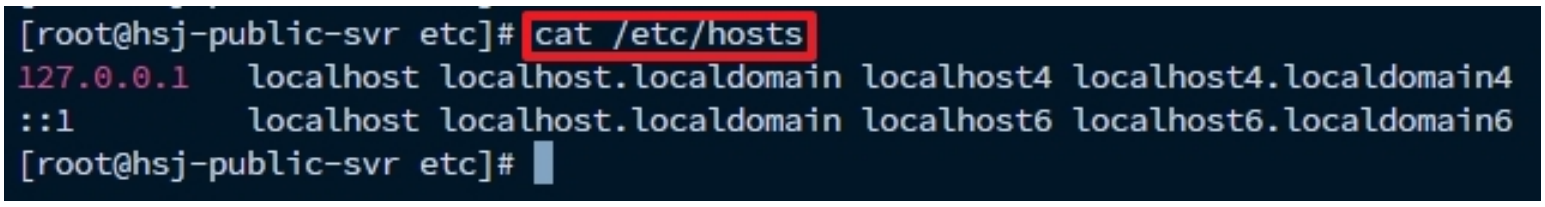
가 정상적인 상태다.

---

4. 현재 /etc/hosts 파일 확인

이제 Linux 서버 내부의 로컬 매핑 파일인 `/etc/hosts`를 확인한다.

```
cat /etc/hosts
```



수정 전 /etc/hosts 파일 내용

현재 파일에는 localhost 관련 항목만 있고 `hwangsoojin.cloud`에 대한 별도 매핑은 없다.

따라서 이 상태에서는 일반적인 이름 해석 과정에서 로컬 파일로 `hwangsoojin.cloud`을 해결할 수 없고, 이후 DNS 조회 단계로 넘어가게 된다.

> 🔎 **중요:** `/etc/hosts` 파일을 읽는 것 자체는 DNS 질의가 아니다.
> 파일은 서버의 로컬 디스크에 있기 때문에 이 과정에는 Ethernet/Wi-Fi 프레임, IP 패킷, TCP/UDP, DNS 메시지가 발생하지 않는다.

---

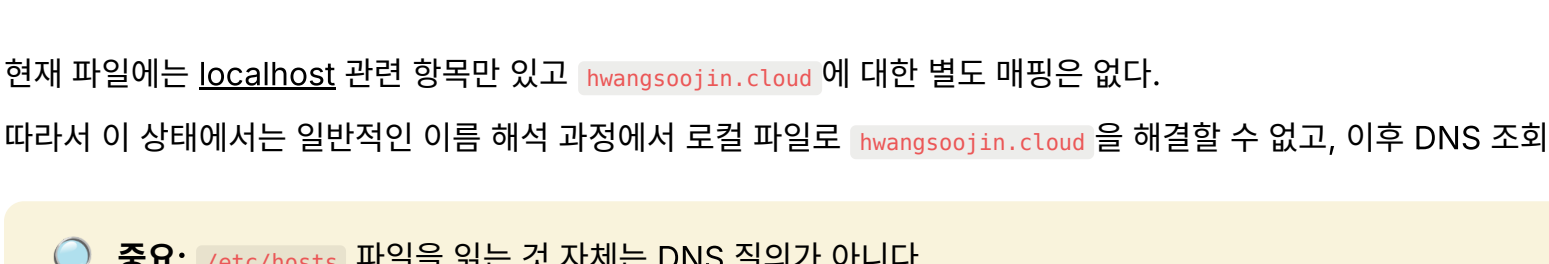
5. 수정 전 hosts 파일 백업

시스템 파일을 수정하기 전에 기존 파일을 백업했다.

```
cp /etc/hosts /etc/hosts.bak
```

그리고 백업이 정상적으로 생성됐는지 확인했다.

```
cat /etc/hosts.bak
```



/etc/hosts를 /etc/hosts.bak으로 백업한 모습

`/etc/hosts`는 시스템 이름 해석에 직접 영향을 줄 수 있으므로 실습이나 운영 작업에서는 수정 전에 백업해 두는 것이 좋다.

---

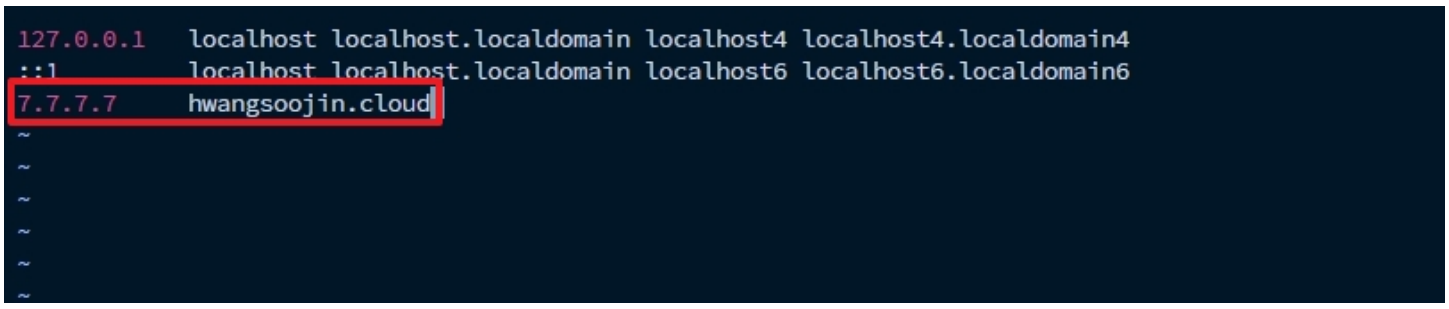
6. /etc/hosts에 임의의 IP 추가

이제 `vi` 편집기로 hosts 파일을 연다.

```
vi /etc/hosts
```

그리고 다음 한 줄을 추가했다.

```
7.7.7.7 hwangsoojin.cloud
```



/etc/hosts에 hwangsoojin.cloud → 7.7.7.7 매핑을 추가한 모습

여기서 `7.7.7.7`은 이번 실습에서 **실제 서비스 목적지가 아닌 임의의 주소로 지정한 값**이다.

중요한 것은 IP 자체가 아니라 다음 매핑이 Linux 서버 내부에 생겼다는 점이다.

```
hwangsoojin.cloud → 7.7.7.7
```

그리고 이것 역시 외부 DNS 서버의 A 레코드를 변경한 것이 아니다.

오직 **이 Linux 서버 한 대의 로컬 이름 해석 결과만 바뀌게 된다.**

---

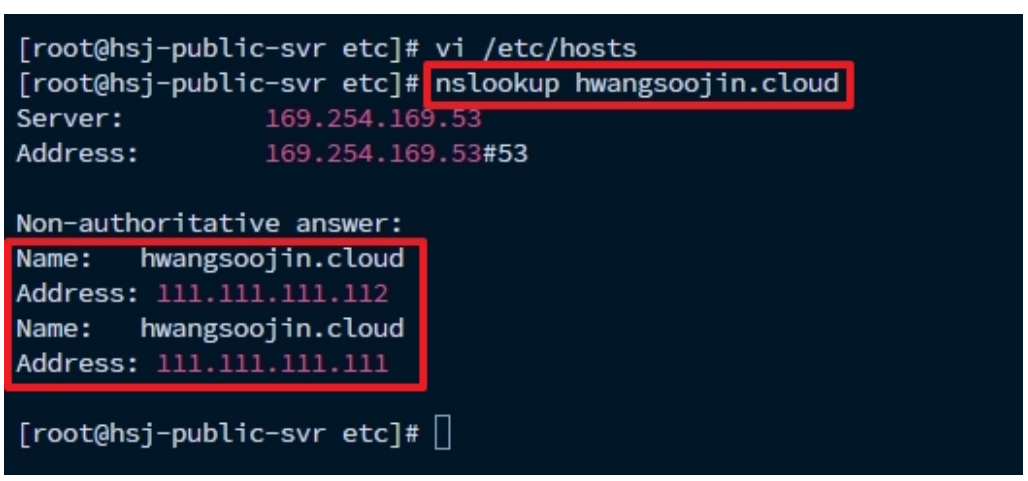
7. 그런데 nslookup 결과는 왜 그대로일까?

hosts 파일을 수정한 뒤 다시 `nslookup`을 실행한다.

```
nslookup hwangsoojin.cloud
```

그런데 결과는 여전히 다음과 같다.

```
111.111.111.111
111.111.111.112
```



/etc/hosts 수정 후에도 nslookup은 기존 DNS A 레코드를 반환하는 모습

처음 보면 `/etc/hosts` 설정이 적용되지 않은 것처럼 보인다.

하지만 **정상적인 결과다.**

`nslookup`은 시스템의 일반적인 이름 해석 경로를 확인하는 명령이 아니라, **DNS 서버에 직접 질의해서 DNS 레코드를 확인하는 도구**다.

따라서

```
/etc/hosts
7.7.7.7 hwangsoojin.cloud
```

라고 적어도 DNS 서버에는 아무 변화가 없으므로 `nslookup`은 계속 실제 DNS 값인

```
111.111.111.111
111.111.111.112
```

를 보여준다.

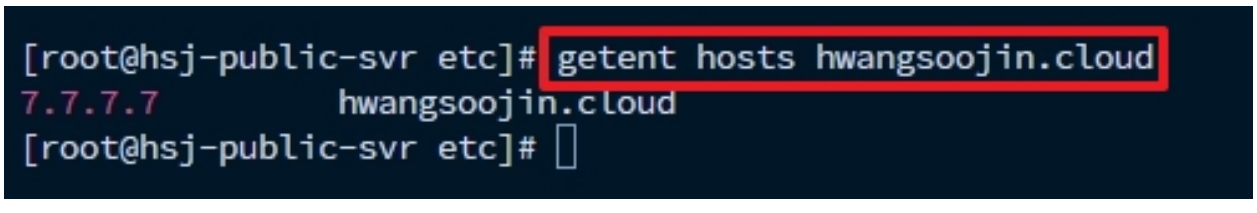
`dig`나 `host` 역시 같은 이유로 hosts 파일 확인 용도로는 적절하지 않다.

---

8. getent hosts로 시스템 이름 해석 결과 확인

이번에는 `getent hosts`를 사용한다.

```
getent hosts hwangsoojin.cloud
```



getent hosts에서 hwangsoojin.cloud가 7.7.7.7로 해석되는 모습

이번에는 우리가 `/etc/hosts`에 입력한 값이 반환된다.

```
7.7.7.7 hwangsoojin.cloud
```

왜 `nslookup`과 결과가 다를까?

`getent`는 Name Service Switch, 즉 **Linux 시스템이 사용하는 이름 해석 체계**를 따른다.

보통 `/etc/nsswitch.conf`에는 다음과 비슷한 설정이 존재한다.

```
hosts: files dns
```

이 경우 의미는 다음과 같다.

```
호스트 이름을 IP로 변환
        ↓
files 확인
        ↓
/etc/hosts 확인
        ↓
여기서 찾으면 해당 IP 사용
        ↓
없으면 DNS 조회
```

즉, `hwangsoojin.cloud`이 이미 `/etc/hosts`에서 발견됐기 때문에 DNS 서버까지 가지 않고 `7.7.7.7`을 사용할 수 있다.

> 정확한 조회 순서는 Linux 배포판과 `/etc/nsswitch.conf` 설정에 따라 달라질 수 있다. 핵심은 **/etc/hosts와 DNS의 우선순위가 시스템 이름 해석 설정에 의해 결정된다**는 점이다.

---

9. ping과 curl로 실제 애플리케이션 관점에서 확인

이번에는 실제 프로그램이 도메인을 어떻게 해석하는지 확인한다.

먼저 `ping`을 실행했다.

```
ping -c 1 hwangsoojin.cloud
```

출력을 보면 `hwangsoojin.cloud`이 다음과 같이 해석된다.

```
hwangsoojin.cloud (7.7.7.7)
```

이어서 `curl`도 실행했다.

```
curl -v --connect-timeout 3 http://hwangsoojin.cloud
```

```
[root@hsj-public-svr etc]# ping -c 1 hwangsoojin.cloud
PING hwangsoojin.cloud (7.7.7.7) 56(84) bytes of data.
^C
--- hwangsoojin.cloud ping statistics ---
1 packets transmitted, 0 received, 100% packet loss, time 0ms

[root@hsj-public-svr etc]# curl -v --connect-timeout 3 http://hwangsoojin.cloud
*   Trying 7.7.7.7:80...
* Connection timed out after 3000 milliseconds
* Closing connection 0
curl: (28) Connection timed out after 3000 milliseconds
[root@hsj-public-svr etc]#
```

ping과 curl에서도 hwangsoojin.cloud가 7.7.7.7로 해석되는 모습

실제 요청은 Timeout이 발생한다.

하지만 이번 실습에서는 **정상적인 결과**다.

`7.7.7.7`을 실제 웹 서버 주소로 지정한 것이 아니기 때문에 응답 자체를 기대한 것이 아니다.

확인하려던 핵심은 이것이다.

```
hwangsoojin.cloud
        ↓
7.7.7.7
```

즉, 일반 애플리케이션 입장에서도 `/etc/hosts`에 추가한 로컬 매핑이 적용되고 있다는 것을 확인했다.

---

10. 실습 후 /etc/hosts 원복

테스트가 끝났으므로 백업해 둔 파일을 이용해 원래 상태로 되돌린다.

```
cp /etc/hosts.bak /etc/hosts
```

그 뒤 파일을 다시 확인했다.

```
cat /etc/hosts
```

```
[root@hsj-public-svr etc]# cp /etc/hosts.bak /etc/hosts
cp: overwrite '/etc/hosts'? y
[root@hsj-public-svr etc]# cat /etc/hosts
127.0.0.1   localhost localhost.localdomain localhost4 localhost4.localdomain4
::1         localhost localhost.localdomain localhost6 localhost6.localdomain6
[root@hsj-public-svr etc]#
```

백업 파일을 이용해 /etc/hosts를 원래 상태로 복구한 모습

이제 `hwangsoojin.cloud → 7.7.7.7` 매핑이 제거됐으므로 다시 시스템의 정상적인 이름 해석 경로를 통해 DNS 결과를 사용하게 된다.

/etc/hosts 조회는 네트워크 통신일까?

이번 실습에서 특히 구분해야 할 부분이다.

`/etc/hosts`에서 도메인을 찾는 과정은 **네트워크 통신이 아니다.**

```
애플리케이션
     ↓
OS 이름 해석 로직
     ↓
/etc/nsswitch.conf 확인
     ↓
/etc/hosts 파일 확인
```

여기까지는 서버 내부에서 파일을 읽는 동작이다.

따라서 이 과정에는

- L2 Ethernet 프레임
- L3 IP 패킷
- L4 TCP/UDP
- L7 DNS 메시지

가 발생하지 않는다.

반면 `/etc/hosts`에서 이름을 해결하지 못하고 실제 DNS 서버에 질의하는 단계로 넘어가면 그때부터 네트워크 통신이 발생한다.

---

nslookup, dig와 getent의 차이

이번 실습을 이해하려면 이 구분이 가장 중요하다.

명령어	주로 확인하는 것	/etc/hosts 반영 확인
`nslookup`	DNS 서버의 응답	적합하지 않음
`dig`	DNS 레코드와 상세 DNS 응답	적합하지 않음
`host`	DNS 기반 이름 조회	적합하지 않음
`getent hosts`	시스템 이름 해석 결과	적합함
`ping`	애플리케이션에서 해석된 목적지 IP	확인 가능
`curl`	실제 애플리케이션의 HTTP 연결 대상	확인 가능

즉,

```
DNS 자체가 무엇을 반환하는지 확인
→ nslookup / dig

이 Linux 서버가 실제로 어떤 IP를 사용할지 확인
→ getent hosts / ping / curl
```

로 구분하면 된다.

/etc/hosts는 언제 유용할까?

클라우드나 서버 운영에서도 자주 활용할 수 있다.

예를 들어 현재 서비스가 기존 Load Balancer를 바라보고 있지만, DNS를 변경하기 전에 신규 Load Balancer를 검증하고 싶다고 하자.

실제 DNS를 바꾸면 모든 사용자에게 영향을 줄 수 있다.

이럴 때 테스트 서버 한 대의 `/etc/hosts`에만 다음과 같이 등록할 수 있다.

```
신규_LB_IP service.example.com
```

그러면 외부 DNS는 그대로 유지한 채 해당 Linux 서버에서만

```
service.example.com
       ↓
신규 Load Balancer
```

로 접속하게 만들 수 있다.

따라서 DNS 전환 전

- 신규 서버 테스트
- Load Balancer 연결 확인
- Reverse Proxy 검증
- 애플리케이션의 Host 기반 동작 확인

- 서비스 전환 사전 점검

등에 활용할 수 있다.

정리

이번 실습에서 실제 DNS는 끝까지 그대로였다.

```
DNS 서버의 실제 응답

hwangsoojin.cloud
   ├─ 111.111.111.111
   └─ 111.111.111.112
```

하지만 Linux 서버의 `/etc/hosts`에 다음 한 줄을 추가했다.

```
7.7.7.7 hwangsoojin.cloud
```

그 결과

```
nslookup / dig
      ↓
DNS 서버 직접 조회
      ↓
111.111.111.111
111.111.111.112
```

반면 일반적인 시스템 이름 해석을 사용하는 쪽에서는

```
getent / ping / curl
        ↓
/etc/hosts 반영
        ↓
7.7.7.7
```

이라는 차이를 직접 확인할 수 있었다.

> ✅ **한 줄 정리**
> Linux에서 `/etc/hosts`는 DNS 서버의 레코드를 바꾸는 기능이 아니라 **해당 서버의 로컬 이름 해석 결과를 바꾸는 파일**이다. 그리고 이를 확인할 때는 `nslookup`이나 `dig`보다 `getent hosts`, `ping`, `curl`처럼 시스템의 일반적인 이름 해석 경로를 사용하는 도구를 봐야 한다.